

1. What is Python?

Python is a **high-level**, interpreted programming language.

Features:

- Easy to learn
- Readable syntax
- Cross-platform

Used in:

- Web development
 - Data science
 - AI / Machine Learning
 - Automation
-

2. Python Syntax

```
x = 10  
print(x)
```

Explanation:

- No need for semicolon
- Indentation is important

Use: Write simple programs

3. Variables

```
a = 10  
name = "John"
```

No need to declare type

Use:

Store data

4. Data Types

```
x = 10      # int
y = 5.5     # float
name = "John" # string
flag = True  # boolean
```

Types:

- int
- float
- str
- bool
- list
- tuple
- dict
- set

Use:

Define data type

5. Type Casting

```
x = int("10")
```

Use: Convert data types

6. Operators

```
sum = 5 + 3
```

Types:

- Arithmetic
- Comparison
- Logical

Use:

Perform operations

7. Input and Output

```
name = input("Enter name: ")  
print(name)
```

Use: User interaction

8. Conditional Statements

```
if x > 10:  
    print("Greater")  
else:  
    print("Smaller")
```

Use:

Decision making

9. Loops

For Loop

```
for i in range(5):  
    print(i)
```

While Loop

```
while x < 5:  
    x += 1
```

Use:

Repeat tasks

10. Strings

```
text = "Hello"  
print(text.upper())
```

Use:

Text operations

11. Lists

```
arr = [1,2,3]  
arr.append(4)
```

Use:

Store multiple values

12. Tuples

```
t = (1,2,3)
```

Immutable list

Use:

Fixed data

13. Sets

```
s = {1,2,3}
```

Unique values

Use:

Remove duplicates

14. Dictionary

```
d = {"name":"John", "age":20}
```

Key-value pairs

Use:

Structured data

15. Functions

```
def greet():  
    print("Hello")
```

Use:

Reusable code

16. Lambda Function

```
add = lambda a,b: a+b
```

Use:

Short function

17. Modules

```
import math  
print(math.sqrt(16))
```

Use:

Use built-in libraries

18. Exception Handling

```
try:  
    x = 10/0  
except:  
    print("Error")
```

Use:

Handle errors

19. File Handling

```
f = open("file.txt","r")  
print(f.read())
```

Use:

Read/write files

20. OOP in Python

```
class Person:  
    def __init__(self,name):  
        self.name = name
```

Use:

Object-oriented programming

21. Inheritance

```
class B(A):  
    pass
```

Use:

Code reuse

22. Polymorphism

Same function different behavior

Use:

Flexible code

23. Encapsulation

Data hiding

Use:

Secure data

24. Abstraction

Hide complexity

Use:

Clean design

25. Iterators

```
for i in arr:  
    print(i)
```

Use:

Loop through data

26. Generators

```
def gen():  
    yield 1
```

Use:

Memory efficient

27. Decorators

```
def my_decorator(func):  
    return func
```

Use:

Modify functions

28. Regular Expressions

```
import re  
re.search("a","apple")
```

Use:

Pattern matching

29. JSON Handling

```
import json  
json.dumps(data)
```

Use:

Data exchange

30. Date and Time

```
import datetime  
print(datetime.datetime.now())
```

Use:

Work with time

31. Database (SQLite)

```
import sqlite3  
conn = sqlite3.connect("db.db")
```

Use:

Store data

32. Virtual Environment

```
python -m venv env
```

Use:

Manage packages

33. Pip (Package Manager)

```
pip install numpy
```

Use:

Install libraries

34. Web Development (Flask)

```
from flask import Flask  
app = Flask(__name__)
```

Use:

Build web apps